ISEL - PIV

Semestre 2024/25

2º Trabalho Laboratorial

Estimação e Classificação de Movimento

Trabalho realizado por:

- João Ramos, numero **50730**
- Fábio Pestana, numero **50756**

Grupo: 07

Turma: 51D

Docente: Pedro Mendes Jorge

Índice

1. Introdução
2. Imports
3. Deteção da mão pelo tom de pele
4. Deteção do movimento da mão
5. Classificação de movimentos
6. Deteção de *zoom in* e *out* da mão
7. Classe do cubo que será animado segundo os movimentos da mão
8. Aplicação
9. Conclusão

Introdução

Este segundo trabalho laboratorial tem como objetivo desenvolver um algoritmo de visão por computador para estimar e classificar movimentos de uma mão capturados por uma sequência de vídeo. O algoritmo será integrado em uma aplicação gráfica interativa desenvolvida com Pygame, permitindo a manipulação em tempo real de um objeto animado (um cubo) com base nos movimentos reconhecidos. A implementação será feita em Python, utilizando a biblioteca OpenCV.
A tarefa principal consiste em identificar seis movimentos distintos realizados com a mão, mapeando-os para ações específicas no objeto animado: mover o cubo para cima, para baixo, para a esquerda, para a direita, aplicar *zoom in* e *zoom out*. O classificador

de movimentos deverá ser robusto, utilizando *frames* consecutivos para verificar a consistência do movimento antes de aplicar a ação correspondente. A aplicação deverá ser capaz de operar tanto em tempo real, utilizando a câmera do *user*, quanto processar vídeos pré-gravados.

Este projeto tem como foco principal explorar as potencialidades da visão computacional para interação entre o utilizador e o computador, proporcionando uma experiência prática com as funcionalidades da biblioteca OpenCV, bem como a integração de algoritmos de processamento de vídeo em aplicações gráficas interativas. Além disso, busca reforçar a capacidade de desenvolver sistemas de reconhecimento em tempo real com eficiência.

Imports

In [64]:
```
import numpy as np
import matplotlib.pyplot as plt
import cv2 as cv
import pygame
from pygame.locals import *
import math
from tkinter import Tk
from tkinter.filedialog import askopenfilename
import datetime
```

Deteção da mão pelo tom de pele

Este bloco de código tem como objetivo detetar regiões de pele em um *frame* de vídeo e destacá-las ao desenhar retângulos ao redor das áreas significativas. A deteção de pele é crucial para posteriormente os movimentos da mão serem detetados e classificados.

Primeiramente, os limites da cor de pele no espaço de cores YCrCb são definidos pelas variáveis `min_YCrCb` e `max_YCrCb`. O espaço de cores YCrCb é frequentemente utilizado para deteção de pele porque separa bem as informações de luminância (Y) e crominância (Cr, Cb).

A linha `imageYCrCb = cv.cvtColor(frame, cv.COLOR_BGR2YCR_CB)` converte o *frame* do espaço de cores BGR (padrão) para YCrCb, para a criação da máscara da pele. Em seguida, a função `cv.inRange(imageYCrCb, min_YCrCb, max_YCrCb)` gera uma máscara binária onde os pixels dentro dos limites definidos são marcados como 1 (255 convertido para binário) e os demais como 0. Isto resulta em uma imagem que destaca apenas as áreas que potencialmente correspondem à pele.

Para reduzir ruídos e melhorar a consistência da máscara, aplicamos uma suavização com a função `cv.GaussianBlur(skinRegion, (5, 5), 0)`. A suavização ajuda a eliminar pequenas imperfeições que podem surgir devido à iluminação no *frame*. Operações morfológicas também são utilizadas para preencher lacunas e suavizar os contornos na máscara. Isto garante que as áreas detetadas formem regiões contínuas.

Por fim, após convertermos a máscara em uma imagem binária, utilizamos a função `cv.findContours` para encontrar os contornos das áreas binárias detetadas como pele. Para cada contorno identificado, calculamos a sua área com `cv.contourArea(contour)` e verificamos se ela é maior ou igual à área mínima definida pelo parâmetro `min_skin_area`. Esta verificação filtra regiões pequenas ou irrelevantes que podem ser causadas por ruídos. Para as regiões que passam no critério, desenhamos um retângulo ao redor delas utilizando `cv.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)`, onde (x, y, w, h) são as dimensões do contorno delimitado.

O *frame* processado com os retângulos desenhados ao redor das áreas de pele é retornado pela função. Este resultado será utilizado em etapas posteriores.

In [ ]:
```python
def detect_skin_and_draw(frame, min_skin_area):
    """
    Deteta regiões de pele em um frame e desenha retângulos ao redor
    das áreas significativas.

    Args:
        - frame (numpy.ndarray) - Frame do vídeo capturado.
        - min_skin_area (int) - Área mínima para considerar uma região
        como pele.

    Returns:
        - numpy.ndarray - Frame processado com regiões de pele
        destacadas.
    """

    # Definir limites de pele em YCrCb
    min_YCrCb = np.array([0, 140, 97], np.uint8)
    max_YCrCb = np.array([255, 173, 127], np.uint8)

    str_elem = cv.getStructuringElement(cv.MORPH_ELLIPSE,
                                        (15, 15), (-1, -1))

    # Converter para o espaço de cor YCrCb
    imageYCrCb = cv.cvtColor(frame, cv.COLOR_BGR2YCR_CB)

    # Criar máscara da área da pele
    skinRegion = cv.inRange(imageYCrCb, min_YCrCb, max_YCrCb)

    blurred = cv.GaussianBlur(skinRegion, (5, 5), 0)

    # Aplicar morfologia para suavizar a máscara
    skinBlurMorph = cv.morphologyEx(blurred, cv.MORPH_CLOSE, str_elem)
    skinRegionMorph = cv.morphologyEx(skinBlurMorph, cv.MORPH_CLOSE,
                                      str_elem)

    _, bin = cv.threshold(skinRegionMorph, 127, 255, cv.THRESH_BINARY)

    # Encontrar contornos das regiões detectadas
    contours, _ = cv.findContours(bin, cv.RETR_EXTERNAL,
                                  cv.CHAIN_APPROX_SIMPLE)

    # Procurar regiões significativas
    for contour in contours:
```

```
        area = cv.contourArea(contour)
        # Apenas processar regiões maiores que a área mínima
        if area >= min_skin_area:
            x, y, w, h = cv.boundingRect(contour)
            cv.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)

    return frame
```

Deteção do movimento da mão

Este bloco de código deteta o movimento de uma região detetada como pele entre dois *frames* consecutivos de vídeo, determinando o deslocamento do centro do retângulo delimitador que envolve a maior área de pele detetada.

Embora utilize técnicas similares à função `detect_skin_and_draw`, como a conversão para o espaço YCrCb, criação de máscaras binárias, aplicação de operações morfológicas e análise de contornos, esta função tem um propósito único: calcular o vetor de movimento (dx, dy) do centro do retângulo entre os dois *frames*.

Para atingir este objetivo, primeiramente, para o primeiro contorno válido encontrado, é desenhado um retângulo ao seu redor, e o centro desse retângulo é calculado como `(x + w // 2, y + h // 2)` (`x` e `y` são as coordenadas do canto superior esquerdo do retângulo, `w` e `h` são a largura e altura do retângulo).

Se um centro válido for detetado, o vetor de deslocamento entre o centro atual e o centro do *frame* anterior é calculado. A posição anterior do centro é armazenada em `track_rectangle_movement.prev_center`. Isto permite que o deslocamento horizontal (dx) e vertical (dy) do retângulo seja determinado. Caso esta seja a primeira execução ou nenhuma região significativa tenha sido detetada, o vetor de deslocamento será (0, 0).

Por fim, a função retorna três valores:

- O vetor de movimento `(dx, dy)`, que representa o deslocamento em pixels do centro do retângulo entre os dois *frames*.
- O *frame* atual processado, com retângulos desenhados ao redor das regiões detetadas como pele.
- O valor da área da maior região detetada como pele, ou 0 caso nenhuma tenha sido encontrada.

In [ ]:
```
def track_rectangle_movement(prev_frame, curr_frame, min_skin_area):
    """
    Deteta o movimento do retângulo que representa o tom de pele entre
    dois frames consecutivos e calcula o vetor de deslocamento do centro
    do retângulo.

    Args:
        - prev_frame (numpy.ndarray) - Frame anterior capturado
        da sequência de vídeo.
```

```
    - curr_frame (numpy.ndarray) - Frame atual capturado
    da sequência de vídeo.
    - min_skin_area (int) - Área mínima para considerar uma
    região como pele.

Returns:
    - tuple - Vetor de movimento (dx, dy) do centro do retângulo
    em pixels. Retorna (0, 0) se não houver movimento.
    - numpy.ndarray - Frame atual processado com retângulos
    desenhados ao redor das regiões detetadas como pele.
    - int - Área da maior região detetada como pele.
    Retorna 0 se nenhuma região foi detetada.
"""

# Definir limites do tom de pele em YCrCb
min_YCrCb = np.array([0, 140, 97], np.uint8)
max_YCrCb = np.array([255, 173, 127], np.uint8)

str_elem = cv.getStructuringElement(cv.MORPH_ELLIPSE, (15, 15))

# Converter frames para YCrCb
prev_YCrCb = cv.cvtColor(prev_frame, cv.COLOR_BGR2YCR_CB)
curr_YCrCb = cv.cvtColor(curr_frame, cv.COLOR_BGR2YCR_CB)

# Criar máscaras de pele
curr_skin_region = cv.inRange(curr_YCrCb, min_YCrCb, max_YCrCb)

blurred = cv.GaussianBlur(curr_skin_region, (5, 5), 0)

curr_blur_region = cv.morphologyEx(blurred, cv.MORPH_CLOSE, str_elem)

curr_skin_morph = cv.morphologyEx(curr_blur_region, cv.MORPH_CLOSE,
                                  str_elem)

_, bin = cv.threshold(curr_skin_morph, 127, 255, cv.THRESH_BINARY)

# Encontrar contornos no frame atual
contours, _ = cv.findContours(bin, cv.RETR_EXTERNAL,
                              cv.CHAIN_APPROX_SIMPLE)

# Inicializar o vetor de movimento
dx, dy = 0, 0
curr_center = None
area = 0
detected_skin = False

# Processar contornos do frame atual
for contour in contours:
    area = cv.contourArea(contour)
    # Apenas regiões significativas
    if area >= min_skin_area:
        x, y, w, h = cv.boundingRect(contour)
        curr_center = (x + w // 2, y + h // 2)
        cv.rectangle(curr_frame, (x, y), (x + w, y + h),
                     (0, 255, 0), 2)
        detected_skin = True
        break  # Usar apenas o primeiro contorno válido

# Calcular deslocamento se tivermos uma posição anterior
if curr_center:
```

```
        if not hasattr(track_rectangle_movement, "prev_center"):
            # Inicializa na primeira execução
            track_rectangle_movement.prev_center = curr_center
        else:
            prev_center = track_rectangle_movement.prev_center
            dx = curr_center[0] - prev_center[0]
            dy = curr_center[1] - prev_center[1]
            # Atualiza posição para o próximo frame
            track_rectangle_movement.prev_center = curr_center

    if not detected_skin:
        area = 0

    return (dx, dy), curr_frame, area
```

Classificação de movimentos

Este bloco de código é responsável por interpretar os deslocamentos horizontais (`dx`) e verticais (`dy`) do centro do retângulo detetado e mover o cubo virtual para a extremidade correspondente da tela. Para tal, comparamos os valores absolutos de `dx` e `dy` com limites predefinidos, identificando a direção predominante do movimento (horizontal ou vertical) e executamos a ação apropriada.

Primeiramente, verificamos se o cubo está em estado de animação através da propriedade `cube.animating` . Caso esteja no processo de animação, a função não retorna nada imediatamente, evitando assim interrupções durante a execução de uma animação.

A largura e altura da tela são calculadas a partir das coordenadas centrais do cubo, assumindo que ele está inicialmente posicionado no meio da tela. Estes valores são usados para determinar as extremidades horizontais e verticais, que representam os possíveis destinos do cubo.

A classificação do movimento é realizada comparando os valores absolutos de `dx` e `dy` .

Se `abs(dx) > abs(dy)` , o movimento predominante é horizontal:

- Um valor positivo de dx superior ao limite `threshold_x` indica um movimento para a direita, o que leva o cubo para a extremidade direita da tela.
- Um valor negativo de dx menor que `- threshold_x` indica um movimento para a esquerda, o que leva o cubo para a extremidade esquerda da tela.

Se `abs(dx) < abs(dy)` , o movimento predominante é vertical:

- Um valor positivo de dy superior ao limite `threshold_y` indica um movimento para baixo, o que leva o cubo para a parte inferior da tela.
- Um valor negativo de dy menor que `- threshold_y` indica um movimento para cima, o que leva o cubo para a parte superior da tela.

A função `cube.start_animation` é então chamada com as coordenadas de destino apropriadas, iniciando o movimento do cubo para o local especificado.

In [ ]:
```python
def classify_movement_and_move_cube(dx, dy, cube, threshold_x,
                                    threshold_y):
    """
    Classifica o movimento com base nos deslocamentos dx e dy e
    move o cubo até a extremidade correspondente.

    Args:
        - dx (int) - Deslocamento horizontal.
        - dy (int) - Deslocamento vertical.
        - cube (Cubo) - Instância do cubo a ser movida.
        - threshold_x (int) - Limite mínimo para deteção de
        movimento horizontal.
        - threshold_y (int) - Limite mínimo para deteção de
        movimento vertical.

    Returns:
        - None - A função altera diretamente o estado do cubo,
          iniciando uma animação se necessário.
    """

    # Ignorar se o cubo estiver em animação
    if cube.animating:
        return

    screen_width, screen_height = cube.center_x * 2, cube.center_y * 2

    # Classificar movimento baseado nos thresholds
    if abs(dx) > abs(dy):  # Movimento predominante é horizontal
        if dx > threshold_x:
            # Movimento para a direita
            cube.start_animation(screen_width - cube.scale,
                                 cube.center_y)
        elif dx < -threshold_x:
            # Movimento para a esquerda
            cube.start_animation(cube.scale, cube.center_y)
    elif abs(dy) > abs(dx):  # Movimento predominante é vertical
        if dy > threshold_y:
            # Movimento para baixo
            cube.start_animation(cube.center_x,
                                 screen_height - cube.scale)
        elif dy < -threshold_y:
            # Movimento para cima
            cube.start_animation(cube.center_x, cube.scale)
```

Deteção de *zoom in* e *out* da mão

Este bloco de código implementa a lógica para detetar e executar operações de *zoom* com base em mudanças no tamanho da área de pele detetada entre dois *frames* consecutivos. O objetivo é determinar se houve uma aproximação ou afastamento da parte do corpo detetada e, com base nisso, acionar um comando de *zoom* no cubo exibido na aplicação.

Primeiramente, a função verifica três condições iniciais:

- Verifica se as áreas detetadas no *frame* anterior (`prev_area`) e no *frame* atual (`current_area`) não são `None` . Isso garante que há dados válidos para análise.
- Verifica se o cubo não está em estado de animação (`cube.animating`). Esta condição impede que múltiplos comandos de *zoom* sejam acionados enquanto o cubo ainda está em movimento.
- Após essas verificações, o temporizador de *cooldown* (`zoom_timer`) é incrementado para contabilizar o tempo desde o último comando de *zoom*.

Posteriormente é realizado o cálculo da razão entre as áreas detetadas nos dois *frames* (`area_ratio = current_area / prev_area`). Esta razão é usada para identificar se houve uma mudança significativa no tamanho da área detetada:

- Se a razão for maior que `1 + zoom_ratio_threshold` , indica um aumento significativo na área (aproximação), e o comando de *zoom in* é ativado.
- Se a razão for menor que `1 - zoom_ratio_threshold` , representa uma redução significativa na área (afastamento), o que resulta num comando de *zoom out*.

Para evitar que comandos de *zoom* sejam disparados em sequência muito rapidamente, a função utiliza um sistema de *cooldown*. Apenas se o `zoom_timer` ultrapassar o valor definido de `zoom_cooldown` , o comando será executado. Após o *zoom*, o temporizador é reiniciado para garantir que o intervalo mínimo entre comandos seja respeitado.

Por fim, a função retorna o valor atualizado do `zoom_timer` , permitindo que o estado do *cooldown* seja mantido.

In [ ]:
```python
def classify_zoom(prev_area, current_area, cube, zoom_ratio_threshold,
                  zoom_timer, zoom_cooldown):
    """
    Lida com a lógica de zoom com base nas áreas detetadas e cooldowns.

    Args:
        - prev_area (float) - Área da região de pele no frame anterior.
        - current_area (float) - Área da região de pele no frame atual.
        - cube (Cubo) - Instância do cubo a ser atualizado.
        - zoom_ratio_threshold (float) - Threshold para detetar mudança
        significativa no tamanho.
        - zoom_timer (int) - Contador de cooldown para permitir zoom.
        - zoom_cooldown (int) - Número mínimo de frames entre zooms.
    Returns:
        - int - Zoom timer atualizado.
    """
    if (prev_area is not None and
        current_area is not None and
        not cube.animating):
```

```
        # Incrementa o timer de cooldown
        zoom_timer += 1

        # Garantir que a divisão é válida
        if prev_area > 0 and current_area > 0:
            area_ratio = current_area / prev_area
            # Apenas permite zoom se o cooldown tiver passado
            if zoom_timer >= zoom_cooldown:
                # Aumento significativo
                if area_ratio > 1 + zoom_ratio_threshold:
                    cube.update_zoom(zoom_in=True)  # Zoom in
                    zoom_timer = 0  # Reinicia o cooldown após o zoom
                # Diminuição significativa
                elif area_ratio < 1 - zoom_ratio_threshold:
                    cube.update_zoom(zoom_in=False)  # Zoom out
                    zoom_timer = 0  # Reinicia o cooldown após o zoom
    return zoom_timer
```

Classe do cubo que será animado segundo os movimentos da mão

A classe Cubo é responsável por representar e manipular um cubo 3D renderizado em um espaço bidimensional. A classe inclui funcionalidades para rotação, *zoom*, animação e desenho na tela, tornando-a o componente visual e interativo principal da aplicação. O cubo é definido pelos seus vértices, arestas e propriedades centrais como posição, escala e estado de animação.

Métodos da classe Cubo:

- `__init__(self, center_x, center_y, initial_scale)` - O construtor inicializa o cubo com o seu centro, escala inicial e outros parâmetros necessários para a animação, *zoom* e renderização. Ele define as coordenadas dos vértices em 3D, configura as arestas e estabelece cores padrão para as faces e arestas;
- `lerp(self, start, end, t)` - Realiza um cálculo para determinar um valor intermediário entre dois valores, `start` e `end`, baseada em um fator `t`;
- `project(self, x, y, z)` - Converte as coordenadas 3D de um vértice para coordenadas 2D, considerando a escala e a posição do cubo na tela;
- `update_rotation(self, speed)` - Incrementa o ângulo de rotação em torno do eixo Y numa velocidade especificada, para assim permitir a rotação do cubo;
- `update_zoom(self, zoom_in)` - Modifica a escala do cubo para realizar *zoom*. Aumenta ou diminui o tamanho com base na variável booleana `zoom_in`, respeitando os limites definidos por `max_zoom_steps` (variável que indica o número máximo de *zooms*);
- `start_animation(self, target_x, target_y)` - Inicia uma animação para mover o cubo da sua posição atual até um *target*;

- `update_animation(self)` - Atualiza o estado da animação do cubo. Esta é dividida em duas etapas: inicialmente o cubo desloca-se na direção do *target* e depois retorna à sua posição inicial (centro da janela);
- `draw(self, screen)` - Desenha o cubo na janela usando a biblioteca `pygame`. Primeiro, aplica a rotação aos vértices e depois projeta-os em 2D. Em seguida, desenha as faces e as arestas, utilizando cores específicas para diferenciá-las.

In [69]:
```
class Cubo:
    """
    Representa um cubo 3D que pode ser rodado, escalado e animado.

    Attributes:
        - center_x (float) - Coordenada X do centro do cubo na janela.
        - center_y (float) - Coordenada Y do centro do cubo na janela.
        - initial_scale (float) - Escala inicial do cubo.
        - scale (float) - Escala atual do cubo, que pode ser alterada
        por zoom.
        - angle_y (float) - Ângulo de rotação do cubo no eixo Y.
        - current_zoom_steps (int) - Passos de zoom realizados
        (-1, 0, +1).
        - max_zoom_steps (int) - Limite de passos de zoom permitido.
        - animating (bool) - Indica se o cubo está atualmente em
        animação.
        - animation_target (tuple) - Ponto alvo para onde o cubo será
        animado.
        - animation_progress (float) - Progresso da animação (0.0 a 1.0).
        - animation_speed (float) - Velocidade da animação.
        - RED (tuple) - Cor vermelha usada para desenhar as faces do
        cubo.
        - BLACK (tuple) - Cor preta usada para desenhar as arestas do
        cubo.
        - vertices (list) - Coordenadas dos vértices do cubo no espaço
        3D.
        - edges (list) - Definem as arestas do cubo.

    Methods:
        - __init__(center_x, center_y, initial_scale) - Inicializa o
        cubo com os valores padrão.
        - lerp(start, end, t) - Realiza a interpolação linear entre
        dois valores.
        - project(x, y, z) - Projeta coordenadas 3D para 2D com base
        na escala e deslocamento.
        - update_rotation(speed) - Atualiza o ângulo de rotação do
        cubo em torno do eixo Y.
        - update_zoom(zoom_in) - Atualiza o zoom do cubo.
        - start_animation(target_x, target_y) - Inicia a animação do
        cubo para um ponto alvo.
        - update_animation() - Atualiza o estado da animação do cubo.
        - draw(screen) - Desenha o cubo na tela.
    """

    def __init__(self, center_x, center_y, initial_scale):
        """
        Inicializa o cubo com os valores padrão.

        Args:
```

```
            - center_x (float) - Coordenada X do centro do cubo.
            - center_y (float) - Coordenada Y do centro do cubo.
            - initial_scale (float) - Escala inicial do cubo.
        """
        self.center_x = center_x
        self.center_y = center_y
        self.offset_x = center_x
        self.offset_y = center_y
        self.initial_scale = initial_scale
        self.scale = initial_scale
        self.angle_y = 0

        # Controle de zoom
        self.current_zoom_steps = 0  # Passos de zoom (-1 a +1)
        # Máximo de passos de zoom em cada direção
        self.max_zoom_steps = 1

        # Animação
        self.animating = False
        self.animation_target = (center_x, center_y)
        self.animation_progress = 0.0
        self.animation_speed = 0.02

        # Cores
        self.RED = (255, 0, 0)
        self.BLACK = (0, 0, 0)

        # Coordenadas do cubo (3D)
        self.vertices = [
            [-1, -1, -1],
            [1, -1, -1],
            [1, 1, -1],
            [-1, 1, -1],
            [-1, -1, 1],
            [1, -1, 1],
            [1, 1, 1],
            [-1, 1, 1],
        ]

        self.edges = [
            (0, 1), (1, 2), (2, 3), (3, 0),
            (4, 5), (5, 6), (6, 7), (7, 4),
            (0, 4), (1, 5), (2, 6), (3, 7)
        ]

    def lerp(self, start, end, t):
        """
        Realiza a interpolação linear entre dois valores.

        Args:
            - start (float) - Valor inicial.
            - end (float) - Valor final.
            - t (float) - Fator de interpolação (0.0 a 1.0).

        Returns:
            - float - Valor interpolado.
        """

        return start + (end - start) * t
```

```python
    def project(self, x, y, z):
        """
        Projeta coordenadas 3D para 2D.

        Args:
            - x (float) - Coordenada X no espaço 3D.
            - y (float) - Coordenada Y no espaço 3D.
            - z (float) - Coordenada Z no espaço 3D.

        Returns:
            - tuple - Coordenadas projetadas no espaço 2D (x2d, y2d).
        """

        # Ajusta o fator de escala baseado na distância
        scale = self.scale / (z + 5)
        x2d = int(x * scale + self.offset_x)
        y2d = int(y * scale + self.offset_y)
        return x2d, y2d

    def update_rotation(self, speed):
        """
        Atualiza o ângulo de rotação do cubo em torno do eixo Y.

        Args:
            - speed (float) - Velocidade da rotação.
        """

        self.angle_y += speed

    def update_zoom(self, zoom_in):
        """
        Atualiza o zoom do cubo.

        Args:
            - zoom_in (bool) - True para dar zoom in;
            False para dar zoom out.
        """

        if zoom_in and self.current_zoom_steps < self.max_zoom_steps:
            self.current_zoom_steps += 1
        elif (not zoom_in and
              self.current_zoom_steps > -self.max_zoom_steps):
            self.current_zoom_steps -= 1

        self.scale = (self.initial_scale *
                      (1.5 ** self.current_zoom_steps))


    def start_animation(self, target_x, target_y):
        """
        Inicia a animação do cubo para um target.

        Args:
            - target_x (float) - Coordenada X do target.
            - target_y (float) - Coordenada Y do target.
        """

        self.animating = True
        self.animation_target = (target_x, target_y)
        self.animation_progress = 0.0
```

```
    def update_animation(self):
        """
        Atualiza o estado da animação do cubo, movendo-o em direção
        ao seu target e de volta ao seu ponto inicial (centro).
        """

        if not self.animating:
            return

        self.animation_progress += self.animation_speed
        if self.animation_progress <= 0.5:

            # Animação para o target
            t = self.animation_progress / 0.5

            self.offset_x = self.lerp(self.center_x,
                                      self.animation_target[0], t)
            self.offset_y = self.lerp(self.center_y,
                                      self.animation_target[1], t)

        elif self.animation_progress <= 1.0:

            # Animação para o ponto inicial do cubo (centro)
            t = (self.animation_progress - 0.5) / 0.5

            self.offset_x = self.lerp(self.animation_target[0],
                                      self.center_x, t)
            self.offset_y = self.lerp(self.animation_target[1],
                                      self.center_y, t)
        else:
            # Fim da animação
            self.animating = False
            self.offset_x, self.offset_y = self.center_x, self.center_y

    def draw(self, screen):
        """
        Desenha o cubo na janela.

        Args:
            - screen (pygame.Surface) - Superfície onde o
            cubo será desenhado.
        """
        # Matriz de rotação
        cos_y, sin_y = math.cos(self.angle_y), math.sin(self.angle_y)

        # Rotacionar e projetar vértices
        transformed_vertices = []
        for x, y, z in self.vertices:
            # Rotação em Y
            x, z = x * cos_y + z * sin_y, -x * sin_y + z * cos_y
            # Projeção
            transformed_vertices.append(self.project(x, y, z))

        # Desenhar faces do cubo
        face_order = [
            (0, 1, 2, 3),  # Face de trás
            (4, 5, 6, 7),  # Face da frente
            (0, 4, 5, 1),  # Face inferior
            (3, 7, 6, 2),  # Face superior
```

```
            (0, 3, 7, 4),  # Face esquerda
            (1, 2, 6, 5),  # Face direita
        ]
        for face in face_order:
            pygame.draw.polygon(screen, self.RED,
                                [transformed_vertices[i] for i in face])

        # Desenhar arestas do cubo
        for edge in self.edges:
            pygame.draw.line(screen, self.BLACK,
                             transformed_vertices[edge[0]],
                             transformed_vertices[edge[1]], 2)
```

Aplicação

Este bloco de código descreve a lógica principal da aplicação, que utiliza a biblioteca `Pygame` para exibir um *feed* de vídeo em tempo real ou gravado, enquanto permite interações com o cubo 3D. A aplicação oferece modos de câmera e vídeo, além de funcionalidades para gravar um vídeo a ser utilizado depois. O cubo animado responde ao movimento detetado na tela e a captura de vídeo da *webcam* ou de um arquivo de vídeo.

O bloco da aplicação começa com a definição de duas funções auxiliares. A primeira, `render_instructions`, é responsável por exibir as instruções de uso na tela. Estas instruções incluem atalhos de teclado para alternar entre modos de operação, iniciar e parar gravações, e sair da aplicação. O texto é renderizado com uma fonte pré-definida e posicionado no canto superior esquerdo da tela. A segunda função, `render_recording_indicator`, exibe o texto "REC" em vermelho na parte superior direita da tela sempre que a gravação de vídeo estiver ativa, indicando visualmente o estado da aplicação.

Após a definição dessas funções, o `Pygame` é inicializado para configurar a interface gráfica, e o sistema de fontes é configurado para garantir a renderização adequada dos textos. Em seguida, a captura de vídeo é iniciada utilizando a *webcam* padrão. Caso a captura não seja bem-sucedida, a aplicação exibe uma mensagem de erro e encerra o loop principal. Além disso, as dimensões do vídeo capturado são obtidas para assegurar que os *frames* sejam ajustados corretamente durante a exibição. A tela do `Pygame` é configurada para funcionar em modo *fullscreen*, utilizando a resolução do monitor do utilizador.

Com a configuração inicial concluída, inicia-se o *loop* principal da aplicação. Este *loop* processa eventos, captura *frames* de vídeo e controla as interações do utilizador. Por exemplo, ao pressionar a tecla 'C', o modo câmera é ativado, e o vídeo da *webcam* é exibido em tempo real. Por outro lado, pressionar a tecla 'V' permite carregar um arquivo de vídeo, iniciando a sua reprodução. Durante o *loop*, o processamento dos *frames* também inclui a deteção de pele, que identifica áreas específicas na imagem e rastreia movimentos.

Quando o utilizador ativa a gravação ao pressionar 'R', um arquivo de vídeo é criado para salvar os *frames* capturados. O nome do arquivo inclui um *timestamp* para diferenciá-lo de outras gravações (nome dos arquivos é o seguinte: video_tp2_[YYYYMMDD]_[HHMMSS], indicando assim a data, horas, minutos e segundos da gravação). A gravação pode ser encerrada com a tecla 'P'.

Além do controle de captura e gravação, a aplicação também utiliza a deteção de movimento para interagir com um cubo animado na tela. Movimentos detetados entre *frames* consecutivos são analisados, e o cubo é deslocado conforme a direção e intensidade do movimento. O *zoom* do cubo é controlado dinamicamente, baseado na variação de área detetada nos movimentos.

Os *frames* capturados são redimensionados para caberem na tela e são convertidos para o formato utilizado pelo `Pygame`. Estes *frames* são então exibidos junto com o cubo animado, as instruções e o indicador de gravação. No final de cada iteração do *loop*, a tela é atualizada para refletir as alterações mais recentes.

Por fim, ao encerrar a aplicação, os recursos são liberados. A captura de vídeo, seja da *webcam* ou de arquivos, é encerrada, e o `Pygame` é encerrado.

In [ ]:
```
def render_instructions():
    """
        Renderiza as instruções na tela.
    """
    instructions = [
        "Pressione 'C' para mudar para a modo câmera",
        "Pressione 'V' para mudar para o modo vídeo",
        "Pressione 'R' para iniciar gravação",
        "Pressione 'P' para parar gravação",
        "Pressione 'Esc' para sair"
    ]

    y_offset = 10
    for line in instructions:
        text = font.render(line, True, (255, 255, 255))
        screen.blit(text, (10, y_offset))
        y_offset += 40


def render_recording_indicator():
    """
        Renderiza o indicador de gravação.
    """
    if record_video:
        rec_text = font.render("REC", True, (255, 0, 0))
        screen.blit(rec_text, (screen_width - 100, 10))

# Inicializa o Pygame
pygame.init()

# Inicializa o sistema de fontes do Pygame
pygame.font.init()

# Fonte padrão, tamanho 36
font = pygame.font.SysFont(None, 36)
```

```python
# Captura de vídeo com a webcam
video_capture = cv.VideoCapture(0)

camera_on = True
video_on = False

# Caminho do arquivo de vídeo
video_file = None

# Objeto de captura para o vídeo
video_capture_file = None

record_video = False
fps = 30

# Número mínimo de frames entre zooms
zoom_cooldown = 15

# Contador de cooldown para zoom
zoom_timer = 30

# Variavel do loop principal
running = True

if not video_capture.isOpened():
    print("Erro: Não foi possível acessar a câmera.")
    running = False
else:
    fps = video_capture.get(cv.CAP_PROP_FPS)
    print(f"FPS da câmera: {fps}")

CAMERA_VIEW_WIDTH = 480
CAMERA_VIEW_HEIGHT = 320

fourcc = cv.VideoWriter_fourcc(*'XVID')
out = None

# Dimensões do vídeo capturado
FRAME_WIDTH = int(video_capture.get(cv.CAP_PROP_FRAME_WIDTH))
FRAME_HEIGHT = int(video_capture.get(cv.CAP_PROP_FRAME_HEIGHT))

# Dimensões da tela do PC
screen_width = pygame.display.Info().current_w
screen_height = pygame.display.Info().current_h

# Criar janela do Pygame em fullscreen
screen = pygame.display.set_mode((screen_width, screen_height),
                                 pygame.FULLSCREEN)
pygame.display.set_caption("T51D-G07_aplicação")

# Variáveis iniciais
prev_frame = None
prev_area = None
current_area = None
cube = Cubo(screen_width // 2, screen_height // 2, initial_scale=200)

# Tempo entre frames em milissegundos
if fps > 1:
    delay = int(1000 / fps)
```

```
    pygame.time.delay(delay)

while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_ESCAPE:
                running = False
            elif event.key == pygame.K_c:  # Modo câmera
                camera_on = True
                video_on = False
                frame = None
                prev_frame = None
                if video_capture_file:
                    video_capture_file.release()
            elif event.key == pygame.K_v:  # Modo vídeo
                # Abre o explorador de arquivos para escolher um vídeo
                Tk().withdraw()  # Ocultar janela principal do Tkinter
                video_file = askopenfilename(filetypes=
                        [("Arquivos de Vídeo", "*.mp4;*.avi;*.mov")])

                if video_file:
                    camera_on = False
                    video_on = True
                    frame = None
                    prev_frame = None
                    video_capture_file = cv.VideoCapture(video_file)
                    video_fps = video_capture_file.get(cv.CAP_PROP_FPS)

            # Ativa o modo de gravação (feed da câmera)
            elif event.key == pygame.K_r:
                if not record_video:
                    record_video = True
                    video_filename = (
                    "video_tp2_" +
                    datetime.datetime.now().strftime('%Y%m%d') + "_" +
                    datetime.datetime.now().strftime('%H%M%S') + ".mp4"
                    )
                    out = cv.VideoWriter(video_filename, fourcc, 30.0,
                                         (FRAME_WIDTH, FRAME_HEIGHT))

            #Caso o modo de gravação estiver ativo, termina-se a gravação
            elif event.key == pygame.K_p:
                if record_video:
                    record_video = False
                    out.release()
                    out = None


    # Capturar frame
    if camera_on:
        ret, frame = video_capture.read()
        if not ret:
            print("Erro ao capturar o frame da câmera.")
            break
    elif video_on and video_capture_file:
        ret, frame = video_capture_file.read()
        if not ret:  # Se o vídeo terminar, reiniciar (faz loop do video)
            video_capture_file.set(cv.CAP_PROP_POS_FRAMES, 0)
```

```
            continue

        frame = cv.resize(frame, (FRAME_WIDTH, FRAME_HEIGHT))

    # Espelha a imagem (para a câmera e video a ser gravado)
    if camera_on:
        frame = cv.flip(frame, 1)
        if record_video:
            out.write(frame)

    # Processar frame para deteção de pele
    skin_frame = detect_skin_and_draw(frame, min_skin_area=600)

    # Rastrear movimento do retângulo
    if prev_frame is not None:
        (dx, dy), processed_frame, current_area = (
            track_rectangle_movement(prev_frame, frame,
                                     min_skin_area=600))

        # Classifica e move o cubo com base no movimento
        classify_movement_and_move_cube(dx, dy, cube,
                    threshold_x=20, threshold_y=20)
    else:
        # Primeiro frame processado
        processed_frame = skin_frame

    # Lógica do zoom
    zoom_timer = classify_zoom(prev_area, current_area,
                cube, zoom_ratio_threshold=0.2,
                zoom_timer=zoom_timer, zoom_cooldown=zoom_cooldown)

    # Atualizar área anterior
    prev_area = current_area

    # Atualizar frame anterior
    prev_frame = frame.copy()

    # Atualizar a rotação e animação do cubo
    cube.update_rotation(0.05)
    cube.update_animation()

    # Redimensiona o frame capturado para as dimensões desejadas
    frame_resized = cv.resize(processed_frame, (CAMERA_VIEW_WIDTH,
                        CAMERA_VIEW_HEIGHT))

    # Converter o frame processado para o formato do Pygame
    frame_rgb = cv.cvtColor(frame_resized, cv.COLOR_BGR2RGB)
    frame_surface = (pygame.surfarray.
            make_surface(frame_rgb.swapaxes(0, 1)))

    # Atualizar a tela do Pygame
    # Fundo preto
    screen.fill((0, 0, 0))

    # Renderizar e desenhar o texto
    # Instruções
    render_instructions()

    # Indicador de gravação
    render_recording_indicator()
```

```
    # Exibir o frame processado
    screen.blit(frame_surface, (0,
            screen_height - frame_resized.shape[0]))

    # Desenhar o cubo animado
    cube.draw(screen)

    # Atualizar a tela do Pygame
    pygame.display.flip()
    pygame.time.delay(delay)

# Liberta os recursos (câmera e ficheiros) e finaliza o Pygame
video_capture.release()
if video_capture_file:
    video_capture_file.release()
if out:
    out.release()
pygame.quit()
```

```
FPS da câmera: 30.0
```

Conclusão

Este trabalho apresenta uma aplicação interativa que integra captura de vídeo em tempo real, processamento de imagem e animações gráficas utilizando as bibliotecas `OpenCV` e `Pygame`. Ao longo do desenvolvimento, foram implementadas funcionalidades como alternância entre diferentes modos de operação (câmera ou vídeo), gravação de *frames* capturados, e a manipulação de um cubo tridimensional que responde a movimentos detetados. A interface gráfica foi projetada para ser intuitiva, com instruções claras e indicadores visuais que orientam o utilizador durante o uso da aplicação.

A deteção e rastreamento de movimentos, bem como a implementação de *zoom* e animações, exemplificam a capacidade de vincular processamento de imagem com elementos gráficos, ampliando as possibilidades de aplicações interativas. Além disso, a atenção ao gerenciamento de recursos, como a captura e liberação de vídeo, garante a estabilidade e eficiência da aplicação.

Como conclusão, este trabalho atingiu os objetivos propostos, apesar de apresentar alguns problemas como a falta de distinção entre mão e face por parte do programa. O projeto tem como potenciais aprimoramentos, a adição de algoritmos mais avançados para deteção de gestos e diferentes partes do corpo e uma interface gráfica ainda mais sofisticada.